

자료구조 실습 보고서

[제 06 주] 재귀 – 성적처리

제출일 : 2018 년 4 월 15 일

201702081 최재범

1. 프로그램 설명서

a. 주요 알고리즘

이번 과제는 재귀를 이용하여 성적을 처리하는 것이며, 배열의 원소 중 최댓값과 최솟값을 구하는 과정과 내림차순으로 정렬하는 과정에서 이를 이용하였다. 문제의 재귀적 풀이는 기본적으로 더 작은 문제로 쪼갬을 반복함으로써 이루어진다.

- `public int partition(int left, int right)`

맨 왼쪽을 pivot 원소로 설정하여, 그 pivot 원소를 기준으로 더 작은 원소는 왼쪽에, 더 큰 원소는 오른쪽에 정렬시킨다.

먼저 pivot 과의 대소관계를 파악하기 위해서는 배열의 원소를 탐색할 인덱스가 필요하다.

인덱스는 왼쪽과 오른쪽에 하나씩 총 두개가 필요한데, 그 이유는 pivot 기준으로 왼쪽에는 작은 원소, 오른쪽에는 큰 원소가 와야 하기 때문에 인덱스를 파악한 후 두 원소를 내림차순으로 정렬하기 위함이다.

왼쪽에서 오른쪽으로 가는 인덱스는 원소를 pivot 과 비교해서 더 크면 인덱스 탐색을 계속하고, 더 작으면 탐색을 멈춘다. 오른쪽에서 왼쪽으로 가는 인덱스는 원소를 pivot 과 비교해서 더 작으면 인덱스 탐색을 계속하고, 더 크면 탐색을 멈춘다.

멈춘 두 인덱스가 교차되어 지나가지 않았을 시에, 두 인덱스에 있는 원소를 교환한다.

이 과정을 전체 인덱스를 순회할 때까지 반복하면, 한 pivot 에 있는 원소에 대하여 전체가 내림차순으로 정렬된다.

정렬 후엔 pivot 이 위치한 인덱스를 반환한다.

- `private void quickSortRecursively(int left, int right)`

왼쪽과 오른쪽의 위치관계가 맞는지 확인하고,

양쪽 구간에 대하여 partition() 을 수행하여 한 pivot 에 대하여 정렬시킨 뒤에,

그 pivot 을 기준으로 둘로 나뉜 양쪽 구간에 대하여 다시 partition() 을 수행한다.

이를 재귀적으로 반복하여 전체를 정렬시킨다.

- `private float sumOfScoresRecursively(int left, int right)`

주어진 구간에 대하여, 맨 왼쪽의 원소 + 그 이후의 구간에 대한 함수의 재귀호출을 이행한다.

left 가 right 보다 커진 후 호출이 되면 0 을 반환하며 이는 탈출조건이 된다.

- `public int maxScoreRecursively(int left, int right)`

주어진 구간에 대하여, 맨 왼쪽의 원소와 그 이후에 구간에 대하여 재귀호출한 값과 비교한 후 더 큰 값을 반환한다. left 와 right 가 같아지면, 그 인덱스의 원소를 반환하며 이는 탈출조건이 된다.

b. 함수 설명서

■ AppController

- public void run() : 프로그램 실행

■ AppView

- public int inputInt() : 정수 입력받아 반환
- public String inputString() : 문자열 입력받아 반환
- public Boolean inputDoesContinueToInputNextStudent() : 다음 점수 입력 여부 확인 후 반환
- public int inputScore() : 점수 입력받아 반환
- public void outputAverageScore(float anAverageScore) : 평균값 출력
- public void outputNumberOfStudentsAboveAverage(int aMaxScore) : 평균 이상인 학생 수 출력
- public void outputMaxScore(int aMaxScore) : 최고 점수 출력
- public void outputMinScore(int aMinScore) : 최저 점수 출력
- public void outputGradeCountFor(char aGrade, int aCount) : 각 학점에 대한 학생 수 출력
- public void outputStudentInfo(int aScore) : 학생들의 점수 출력
- public void outputMessage(String aMessageString) : 문자열 출력

■ Ban

- public int capacity() : 최대 학생 수 반환 (Getter)
- public int size() : 현재 학생 수 반환 (Getter)
- public boolean isEmpty() : 현재 반이 비어 있는지 확인 후 반환
- public boolean isFull() : 현재 반이 꽉 찼는지 확인 후 반환
- public boolean add(Student aStudent) : 학생 추가 후 성공 여부 반환
- public Student elementAt(int anOrder) : 주어진 위치의 학생 객체 반환
- public void sortStudentByScore() : 학생들의 성적 정렬
- public int minScore() : 최저 점수 반환
- public int maxScore() : 최고 점수 반환
- public float averageScore() : 평균 점수 반환
- public int numberOfStudentsAboveAverage() : 평균 이상 학생 수 반환
- public GradeCounter countGrades() : 학점 별 학생 수를 포함하는 GradeCounter 객체 반환

■ GradeCounter

- public void count(char aGrade) : 주어진 학점에 따라 숫자를 셈
- public int numberOfA() : A 학점의 학생 수 반환 (Getter)
- public int numberOfB() : B 학점의 학생 수 반환 (Getter)

- public int numberOfC() : C 학점의 학생 수 반환 (Getter)
- public int numberOfD() : D 학점의 학생 수 반환 (Getter)
- public int numberOfF() : F 학점의 학생 수 반환 (Getter)

■ Student

- public int score() : 학생 점수 반환 (Getter)
- public void setScore(int newScore) : 학생 점수 설정 (Setter)

c. 종합 설명서

입력 여부를 확인받은 뒤에, 점수를 입력받아 저장한다.

종료하면 평균 점수, 평균 이상의 학생 수, 최고/최저 점수, 학점 별 학생 수, 성적순 정렬 결과를 출력한다.

2. 프로그램 장단점 분석

장점 : X

단점 : 예외처리를 사용하지 않아, 숫자를 입력해야 할 때 문자를 입력하면 예외가 발생한다.

3. 실행 결과 분석

a. 입력과 출력

```
<< 성적 처리를 시작합니다. >>
[*] 성적을 입력하려면 'Y' 또는 'y'를, 종료하려면 다른 아무 키나 치시오 : y
- 점수를 입력하시오 : 82
[*] 성적을 입력하려면 'Y' 또는 'y'를, 종료하려면 다른 아무 키나 치시오 : y
- 점수를 입력하시오 : 45
[*] 성적을 입력하려면 'Y' 또는 'y'를, 종료하려면 다른 아무 키나 치시오 : y
- 점수를 입력하시오 : 93
[*] 성적을 입력하려면 'Y' 또는 'y'를, 종료하려면 다른 아무 키나 치시오 : y
- 점수를 입력하시오 : 102
[!] ERROR : 0 보다 작거나 100 보다 커서, 정상적인 점수가 아닙니다.
[*] 성적을 입력하려면 'Y' 또는 'y'를, 종료하려면 다른 아무 키나 치시오 : y
- 점수를 입력하시오 : 66
[*] 성적을 입력하려면 'Y' 또는 'y'를, 종료하려면 다른 아무 키나 치시오 : y
- 점수를 입력하시오 : 87
[*] 성적을 입력하려면 'Y' 또는 'y'를, 종료하려면 다른 아무 키나 치시오 : y
- 점수를 입력하시오 : -1
[!] ERROR : 0 보다 작거나 100 보다 커서, 정상적인 점수가 아닙니다.
[*] 성적을 입력하려면 'Y' 또는 'y'를, 종료하려면 다른 아무 키나 치시오 : n
[*] 성적 입력을 종료합니다.

평균 점수 : 74.6
평균 이상 학생들의 수는 3 명 입니다.
최고 점수는 93 점 입니다.
최저 점수는 45 점 입니다.
A 학점 학생의 수는 1 명 입니다.
B 학점 학생의 수는 2 명 입니다.
C 학점 학생의 수는 0 명 입니다.
D 학점 학생의 수는 1 명 입니다.
F 학점 학생의 수는 1 명 입니다.

[*] 학생들의 성적 순 목록
학생의 점수는 93 점 입니다.
학생의 점수는 87 점 입니다.
학생의 점수는 82 점 입니다.
학생의 점수는 66 점 입니다.
학생의 점수는 45 점 입니다.
<< 성적 처리를 종료합니다. >>
```

b. 결과분석

82, 45, 93, 102, 66, 87, -1 의 7 개 점수를 입력하였고, 102 와 -1 은 잘못된 입력이므로 무시되며 나머지 5 개 점수가 내림차순으로 정렬되었다.

평균인 74.6 이상인 점수는 82, 87, 93 으로 3 개가 맞고, 정렬 결과를 보면 최고점과 최저점이 맞다는 것을 확인할 수 있다.

A : 93 B : 87, 82 C : 없음 D : 66 F : 45

이므로 학점 별 학생 수도 일치한다.

4. 소스코드

AppController.java

```
public class AppController {

    private AppView _appView;
    private Ban _ban;

    // 생성자
    public AppController() {
        this._appView = new AppView();
    }

    public void run() {

        this.showMessage(MessageID.Notice_StartProgram);

        // 1. 성적 입력
        this.inputAndStoreStudents();

        if(this._ban.isEmpty()) {
            this.showMessage(MessageID.Error_NoInputScores);
        }

        else {
            // 2. 정보 출력
            this.showStatistics();

            // 3. 성적순 정렬 후 출력
            this._ban.sortStudentsByScore();
            this.showStudentsSortedByScore();
        }

        this.showMessage(MessageID.Notice_EndProgram);
    }
}
```

```

// 비공개 함수
// 학생들의 정보 입력받아 Ban에 저장
private boolean inputAndStoreStudents() {

    int score;
    boolean storingASuccessful = true;

    this._ban = new Ban();

    // 새로운 입력을 받을 지 확인 후 진행
    while(storingASuccessful &&
        this._appView.inputDoesContinueToInputNextStudent()) {

        score = this._appView.inputScore();

        if(score < 0 || score > 100) {
            this.showMessage(MessageID.Error_InvalidScore);
        }
        else {
            Student aStudent = new Student(score);
            this._ban.add(aStudent);
        }
    }

    this.showMessage(MessageID.Notice_EndMenu);
    return storingASuccessful;
}

// 상태 정보 출력
private void showStatistics() {

    // 평균, 평균 이상 학생수, 최고/최저 출력
    this._appView.outputAverageScore(this._ban.averageScore());
    this._appView.outputNumberOfStudentsAboveAverage
        (this._ban.numberofStudentsAboveAverage());
    this._appView.outputMaxScore(this._ban.maxScore());
    this._appView.outputMinScore(this._ban.minScore());

    // 학점 별 학생 수 출력
    GradeCounter gradeCounter = this._ban.countGrades();
    this._appView.outputGradeCountFor('A', gradeCounter.numberOfA());
    this._appView.outputGradeCountFor('B', gradeCounter.numberOfB());
    this._appView.outputGradeCountFor('C', gradeCounter.numberOfC());
    this._appView.outputGradeCountFor('D', gradeCounter.numberOfD());
    this._appView.outputGradeCountFor('F', gradeCounter.numberOfF());

}

// 성적 순 정렬한 학생 정보 출력
private void showStudentsSortedByScore() {
    this.showMessage(MessageID.Show_SortedStudentList);

    for(int position = 0; position < this._ban.size(); position++) {
        this._appView.outputStudentInfo
            (this._ban.elementAt(position).score());
    }
}

```

```

// 상황에 맞는 메시지 출력
private void showMessage(MessageID aMessageID) {
    switch(aMessageID) {

        case Notice_StartProgram:
            this._appView.outputMessage("<< 성적 처리를 시작합니다. >> \n");
            break;
        case Notice_EndProgram:
            this._appView.outputMessage("<< 성적 처리를 종료합니다. >> \n");
            break;
        case Notice_StartMenu:
            this._appView.outputMessage("\n[*] 성적을 입력하려면 'Y' 또는 'y'를,
종료하려면 다른 아무 키나 치시오 : ");
            break;
        case Notice_EndMenu:
            this._appView.outputMessage("[*] 성적 입력을 종료합니다. \n\n");
            break;

        case Show_SortedStudentList:
            this._appView.outputMessage("\n[*] 학생들의 성적 순 목록 \n");
            break;

        case Error_WrongMenu:
            this._appView.outputMessage("");
            break;
        case Error_InvalidScore:
            this._appView.outputMessage("[!] ERROR : 0 보다 작거나 100 보다 커서,
정상적인 점수가 아닙니다.\n");
            break;
        case Error_NoInputScores:
            this._appView.outputMessage("[!] 성적이 입력되지 않았습니다.\n");
            break;

    }
}
}
}

```

AppView.java

```

import java.util.Scanner;

public class AppView {

    private Scanner _scanner;

    // 생성자
    public AppView() {
        this._scanner = new Scanner(System.in);
    }

    // 입력
    // 정수 입력받아 반환
    public int inputInt() {
        return Integer.parseInt(this._scanner.nextLine());
    }
}

```

```

    }

    // 문자열 입력받아 반환
    public String inputString() {
        return this._scanner.nextLine();
    }

    // 다음 점수 입력 여부 확인 후 반환
    public boolean inputDoesContinueToInputNextStudent() {
        char answer;

        System.out.print("[*] 성적을 입력하려면 'Y' 또는 'y'를, 종료하려면 다른 아무
키나 치시오 : ");
        answer = this.inputString().charAt(0);

        if(answer == 'Y' || (answer == 'y')) {
            return true;
        }
        else {
            return false;
        }
    }

    // 점수 입력받아 반환
    public int inputScore() {
        int score;

        System.out.print("- 점수를 입력하시오 : ");
        score = this.inputInt();
        return score;
    }

    // 출력
    // 평균값 출력
    public void outputAverageScore(float anAverageScore) {
        System.out.println("평균 점수 : " + anAverageScore);
    }

    // 평균 이상인 학생 수 출력
    public void outputNumberOfStudentsAboveAverage(int aNumber) {
        System.out.println("평균 이상 학생들의 수는 " + aNumber + " 명 입니다.");
    }

    // 최고점 출력
    public void outputMaxScore(int aMaxScore) {
        System.out.println("최고 점수는 " + aMaxScore + " 점 입니다.");
    }

    // 최저점 출력
    public void outputMinScore(int aMinScore) {
        System.out.println("최저 점수는 " + aMinScore + " 점 입니다.");
    }
}

```



```

// 각 학점에 대한 학생 수 출력
public void outputGradeCountFor(char aGrade, int aCount) {
    System.out.println(aGrade + " 학점 학생의 수는 " + aCount + " 명 입니다.");
}

// 학생들의 점수 출력
public void outputStudentInfo(int aScore) {
    System.out.println("학생의 점수는 " + aScore + " 점 입니다.");
}

// 문자열 출력
public void outputMessage(String aMessageString) {
    System.out.print(aMessageString);
}
}

```

Ban.java

```

public class Ban {

    public static final int DEFAULT_CAPACITY = 100;           // 기본 최대 학생 수

    private int _capacity;
    private int _size;
    private Student[] _elements;

    // 생성자
    public Ban() {
        this._capacity = Ban.DEFAULT_CAPACITY;
        this._size = 0;
        this._elements = new Student[this._capacity];
    }
    public Ban(int givenCapacity) {
        this._capacity = givenCapacity;
        this._size = 0;
        this._elements = new Student[this._capacity];
    }

    // Getter
    public int capacity() {
        return this._capacity;
    }
    public int size() {
        return this._size;
    }

    // 학급이 비어있거나 꽉 찼는지 확인
    public boolean isEmpty() {
        return this._size == 0;
    }
    public boolean isFull() {
        return this._size >= this._capacity;
    }
}

```

```

}

// 학생 추가
public boolean add(Student aStudent) {
    if( this.isFull() ) {
        return false;
    }
    else {
        this._elements[this._size] = aStudent;
        this._size++;
        return true;
    }
}

// 주어진 위치의 학생 객체 반환
public Student elementAt(int anOrder) {
    return this._elements[anOrder];
}

// 학생들의 성적 정렬
public void sortStudentsByScore() {
    int size = this._size;    // 정렬 : [0] ~ [size - 1]

    // 2 개부터 정렬 가능
    if(size >= 2) {

        // 최솟값 위치 찾기
        int minLoc = 0;
        for(int i = 1; i < size; i++) {
            if(this._elements[i].score() < this._elements[minLoc].score())
            {
                minLoc = i;
            }
        }

        // 최솟값을 구간의 맨 오른쪽으로
        swap(minLoc, size - 1);

        // 정렬
        this.quickSortRecursively(0, size - 2);
    }
}

// 최저점 반환
public int minScore() {
    return this.minScoreRecursively(0, this._size - 1);
}

// 최고점 반환
public int maxScore() {
    return this.maxScoreRecursively(0, this._size - 1);
}

// 평균점 반환

```

```

    public float averageScore() {
        return (float)this.sumOfScoresRecursively(0, this._size - 1) /
(float)this._size;
    }

    // 평균 이상 학생 수 반환
    public int numberOfStudentsAboveAverage() {
        float average = this.averageScore();
        float score;
        int numberOfStudentsAboveAverage = 0;

        for(int i = 0; i < this._size; i++) {
            score = this._elements[i].score();
            if(score >= average) {
                numberOfStudentsAboveAverage++;
            }
        }
        return numberOfStudentsAboveAverage;
    }

    // 학점 별 학생 수 세서 반환
    public GradeCounter countGrades() {
        char currentGrade;
        GradeCounter gradeCounter = new GradeCounter(); // 학점에 따라
        분류하여 카운트하는 객체

        for(int i = 0; i < this._size; i++) {
            currentGrade = this.scoreToGrade((this._elements[i].score()));
            gradeCounter.count(currentGrade);
        }

        return gradeCounter;
    }

    // 비공개 함수
    // 두 학생 위치 교환
    private void swap(int positionA, int positionB) {
        Student temp = this._elements[positionA];
        this._elements[positionA] = this._elements[positionB];
        this._elements[positionB] = temp;
    }

    // QuickSort
    private void quickSortRecursively (int left, int right) {
        if(left < right) {
            int mid = this.partition(left, right); // pivot 정렬
            this.quickSortRecursively(left, mid - 1); // pivot 왼쪽 정렬
            this.quickSortRecursively(mid + 1, right); // pivot 오른쪽
        }
    }
}

```

```

// pivot 기준으로 정렬 : 내림차순
private int partition(int left, int right) {

    int pivotIndex = left;                                // 맨 왼쪽 원소를
pivot으로 설정

    int moveToRightIndex = left + 1;                      // pivot 이후부터 탐색
    int moveToLeftIndex = right;

    // 탐색 사이클이 다 돌지 않았을 때
    while(moveToRightIndex <= moveToLeftIndex) {

        // 왼쪽 탐색 인덱스에 있는 원소가 pivot 보다 클 때 : 가만히 두고 인덱스
이동

        while(this._elements[moveToRightIndex].score()
                > this._elements[pivotIndex].score()) {
            moveToRightIndex++;
        }

        // 오른쪽 탐색 인덱스에 있는 원소가 pivot 보다 작을 때 : 가만히 두고 인덱스
이동

        while(this._elements[moveToLeftIndex].score()
                < this._elements[pivotIndex].score()) {
            moveToLeftIndex--;
        }

        // 찾은 인덱스에 존재하는 원소 끼리의 교환 : 정렬
        if(moveToRightIndex < moveToLeftIndex) {
            this.swap(moveToRightIndex, moveToLeftIndex);
        }

    }

    // pivot 기준 정렬 완료 후, pivot을 맞는 위치로 이동
    // moveToLeftIndex에 있는 원소는 pivot에 있는 원소보다 작으므로,
    // 원래 pivot이 있던 위치인 맨 왼쪽으로 옮겨도 상관 x
    this.swap(moveToLeftIndex, pivotIndex);

    return moveToLeftIndex;                                // swap 후, pivot이 있는 위치
}

// 모든 점수들의 합 반환
private float sumOfScoresRecursively(int left, int right) {
    if(left > right) {
        return 0;
    }
    return this._elements[left].score() + this.sumOfScoresRecursively(left + 1,
right);
}

// 최고점 반환
private int maxScoreRecursively(int left, int right) {
    if(left == right) {
        return this._elements[left].score();
    }
}

```

```

    }
    else {
        return this._elements[left].score() > this.maxScoreRecursively(left +
1, right) ?
        this._elements[left].score() : this.maxScoreRecursively(left
+ 1, right);
    }
}

// 최저점 반환
private int minScoreRecursively(int left, int right) {
    if(left == right) {
        return this._elements[left].score();
    }
    else {
        return this._elements[left].score() < this.minScoreRecursively(left +
1, right) ?
        this._elements[left].score() : this.minScoreRecursively(left
+ 1, right);
    }
}

// 성적을 학점으로 반환
private char scoreToGrade(int aScore) {
    if(aScore >= 90) {
        return 'A';
    }
    else if(aScore >= 80) {
        return 'B';
    }
    else if(aScore >= 70) {
        return 'C';
    }
    else if(aScore >= 60) {
        return 'D';
    }
    else {
        return 'F';
    }
}
}

```

Student.java

```

public class Student {

    private int _score;

    // 생성자
    public Student() {

    }
    public Student(int givenScore) {
        this._score = givenScore;
    }

    // Getter
    public int score() {

```

```

        return this._score;
    }

    // Setter
    public void setScore(int newScore) {
        this._score = newScore;
    }
}

```

GradeCounter.java

```

// 각 학점별 학생 수

public class GradeCounter {

    private int _numberOfA;
    private int _numberOfB;
    private int _numberOfC;
    private int _numberOfD;
    private int _numberOfF;

    // 생성자
    public GradeCounter() { }

    // 학점을 받아서, 해당 학점의 학생 수 증가
    public void count(char aGrade) {
        switch (aGrade) {
            case 'A':
                this._numberOfA++;
                break;
            case 'B':
                this._numberOfB++;
                break;
            case 'C':
                this._numberOfC++;
                break;
            case 'D':
                this._numberOfD++;
                break;
            default:
                this._numberOfF++;
        }
    }

    // Getter : 각 학점의 학생 수 얻기
    public int numberOfA() {
        return this._numberOfA;
    }
    public int numberOfB() {
        return this._numberOfB;
    }
    public int numberOfC() {
        return this._numberOfC;
    }
    public int numberOfD() {
        return this._numberOfD;
    }
    public int numberOfF() {

```

```
        return this._numberOff;  
    }  
}
```

MessageID.java

```
public enum MessageID {  
  
    // Message IDs for Notices :  
    Notice_StartProgram,  
    Notice_EndProgram,  
    Notice_StartMenu,  
    Notice_EndMenu,  
  
    // Message IDs for Shows :  
    Show_SortedStudentList,  
  
    // Message IDs for Errors  
    Error_WrongMenu,  
    Error_InvalidScore,  
    Error_NoInputScores  
}
```

DS1_06_201702081_최재범.java

```
public class DS1_06_201702081_최재범 {  
  
    public static void main(String[] args) {  
  
        ApplicationController appController = new ApplicationController();  
        appController.run();  
  
    }  
}
```